FRONTEND ROADMAP

DAY 14

State Management with Redux

When useState isn't enough. Learn why apps need a central store, set up Redux Toolkit step by step, build a counter, and make the state survive a refresh.

WHAT WE COVER

- Why global state
- Redux vs useState
- Store & slices
- Provider
- Typed hooks
- Persistence

You already manage state inside a component with useState. But what happens when many components — scattered across the app — need the same data, like a shopping cart or the logged-in user? Passing props through every layer gets painful fast. Redux gives you one central store that any component can read from and update. Today we'll learn when that's worth it, set up Redux Toolkit, build a counter, and persist it to localStorage.

State Management with Redux

useState is local to one component. **Redux** is a single, app-wide store that any component can read and update directly — no prop-drilling. With Redux Toolkit the setup is small and fully typed.

CLASS AGENDA · ~2 HOURS

0:00	The problem: prop drilling & shared state	§1
0:20	Redux vs useState; when you need Redux	§2-3
0:40	Install Redux Toolkit & create the store	§4
0:55	Slices, the Provider & typed hooks	§5-7
1:25	Build the counter project	§8
1:45	Persist to localStorage & wrap-up	§9

ON THIS HANDOUT

PART 1 · Why State Management?

01 The problem: prop drilling

Imagine a shopping cart. The navbar shows the item count, the product page has “Add to cart” buttons, and the cart page lists everything. All three are in **different parts** of the component tree, but they all need the **same** cart data.

With only `useState`, the cart would have to live in a common ancestor and be passed **down through every layer** as props — even through components that don't care about it. This is called **prop drilling**, and it makes code tangled and hard to change.

- Many far-apart components need to read and update the same data.
- Threading props through every intermediate component is tedious and fragile.
- We want shared data to live in **one place** that any component can reach directly.

02 Redux vs useState

Both hold state — the difference is **scope and ownership**.

- **useState** — **local** state owned by one component. Perfect for a form field, a toggle, a single counter.
- **Redux** — a **global** store owned by the whole app. Any component can read it and dispatch updates, no matter where it sits.
- useState is per-component and disappears when the component unmounts; the Redux store lives for the whole app session.

Think of useState as a sticky note on one desk, and Redux as a shared whiteboard the whole team can see and update.

03 When you actually need Redux

Redux is powerful but not always necessary. Reach for it when:

- Several unrelated components need the **same** data (cart, current user, theme, notifications).
- State updates are **complex** or need to be predictable and traceable.
- You want great **debugging** — Redux DevTools let you inspect every action and even time-travel.
- The app is **large** and passing props around has become unmanageable.

For small apps, `useState` and React Context are often enough — don't add Redux just because. But for something like an e-commerce cart shared across pages, it's exactly the right tool (which is why we're learning it before Day 15).

PART 2 · Redux Toolkit Setup

04 Setup: the store

We use **Redux Toolkit** — the official, modern way to use Redux (far less boilerplate than old Redux). Install it with React bindings:

TERMINAL

```
npm install @reduxjs/toolkit react-redux
```

The **store** is the single object that holds all your app state. You build it from one or more **reducers** (each managing a slice of the state):

```ts
// store/store.ts
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "./counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer,   // state.counter is managed by this reducer
  },
});

// types we'll use for fully-typed hooks (Day 8-9 pays off here)
export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

- `configureStore` creates the store and wires in your reducers.
- `RootState` and `AppDispatch` are inferred types — we'll use them so selectors and dispatch are type-safe.

05 Slices: state + reducers

A **slice** bundles a piece of state with the functions that update it. `createSlice` generates the **actions** for you automatically.

```ts
// store/counterSlice.ts
import { createSlice, PayloadAction } from "@reduxjs/toolkit";

export interface CounterState { value: number; }
const initialState: CounterState = { value: 0 };

const counterSlice = createSlice({
  name: "counter",
  initialState,
  reducers: {
    increment: (state) => { state.value += 1; },              // looks like mutation...
    decrement: (state) => { state.value -= 1; },
    reset:     (state) => { state.value = 0; },
    incrementByAmount: (state, action: PayloadAction<number>) => {
      state.value += action.payload;                          // action.payload is typed
    },
  },
});

export const { increment, decrement, reset, incrementByAmount } = counterSlice.actions;
export default counterSlice.reducer;
```

- `name` labels the slice; `initialState` is where it starts.
- Each reducer describes how the state changes for one action.
- It looks like you're mutating `state`, but Redux Toolkit uses Immer under the hood to make a safe, immutable copy for you.
- `PayloadAction<number>` types the data an action carries (the `payload`).

06 Providing the store

Wrap your app in a `<Provider>` so every component can reach the store — do this once, at the top.

```tsx
// main.tsx
import { Provider } from "react-redux";
import { store } from "./store/store";
import App from "./App";

export default function Root() {
  return (
    <Provider store={store}>
      <App />
    </Provider>
  );
}
```

Now any component inside `App`, at any depth, can access the store — no prop drilling.

07 Typed hooks

React-Redux gives two hooks: `useSelector` to **read** state and `useDispatch` to **send** actions. Wrap them once with your types so they're fully type-safe everywhere.

```ts
// store/hooks.ts
import { useDispatch, useSelector } from "react-redux";
import type { RootState, AppDispatch } from "./store";

export const useAppDispatch = () => useDispatch<AppDispatch>();
export const useAppSelector = <T,>(selector: (state: RootState) => T): T =>
  useSelector(selector);
```

- `useAppSelector` reads a piece of state, e.g. `state.counter.value`, and knows its type.
- `useAppDispatch` sends actions to the store to change state.

PART 3 · Build & Persist

08 Project: a Redux counter

The classic first Redux app. The counter state lives in the store, so **any** component could read or change it — not just this one.

```tsx
// Counter.tsx
import { useAppSelector, useAppDispatch } from "./store/hooks";
import { increment, decrement, reset, incrementByAmount } from "./store/counterSlice";

export function Counter() {
  const count = useAppSelector((state) => state.counter.value);   // READ
  const dispatch = useAppDispatch();                              // WRITE

  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => dispatch(decrement())}>-1</button>
      <button onClick={() => dispatch(increment())}>+1</button>
      <button onClick={() => dispatch(incrementByAmount(5))}>+5</button>
      <button onClick={() => dispatch(reset())}>Reset</button>
    </div>
  );
}
```

- `useAppSelector` subscribes to the value — the component re-renders when it changes.
- `dispatch(increment())` sends an action; the slice's reducer updates the store.
- Data flow: **dispatch action → reducer updates store → selector re-renders UI**.

> **TRY IT YOURSELF**
>
> Add a text input and an “Add amount” button that dispatches `incrementByAmount` with the typed value.

09 Persistence with localStorage

By default the store resets on refresh. To make it **persist**, save the state to `localStorage` whenever it changes, and load it back when the app starts. A lightweight adapter, no extra libraries:

```ts
// store/store.ts (with persistence)
import { configureStore } from "@reduxjs/toolkit";
import counterReducer, { CounterState } from "./counterSlice";

// 1) load saved state on startup
function loadCounter(): CounterState | undefined {
  try {
    const saved = localStorage.getItem("counter");
    return saved ? (JSON.parse(saved) as CounterState) : undefined;
  } catch {
    return undefined;   // ignore corrupt / missing data
  }
}

const preloaded = loadCounter();

export const store = configureStore({
  reducer: { counter: counterReducer },
  preloadedState: preloaded ? { counter: preloaded } : undefined,   // 2) seed the store
});

// 3) save to localStorage on every change
store.subscribe(() => {
  localStorage.setItem("counter", JSON.stringify(store.getState().counter));
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
```

- `preloadedState` seeds the store from whatever was saved before.
- `store.subscribe(...)` runs after every change — we write the latest state to localStorage.
- Now the counter survives a page refresh. The same adapter works for a cart, theme, or auth token.
- For bigger apps, the `redux-persist` library automates this — but this hand-rolled version shows exactly what it does.

Practice Challenges

Take Redux further with a few of these:

- Add a `step` to the counter state and buttons to change it, then increment by `step`.
- Create a second slice (e.g. `theme` with light/dark) and add it to the store.
- Install the Redux DevTools browser extension and watch actions fire as you click.
- Build a tiny todo slice with `addTodo` / `toggleTodo` / `removeTodo`.
- Persist the todo slice to localStorage with the same adapter pattern.

Conclusion & what's next

You now understand **why** apps need central state, how Redux differs from **useState**, and when it's worth it. You set up **Redux Toolkit** — store, slices, Provider, typed hooks — built a working counter, and made it persist across refreshes.

Next class (Day 15): the **capstone** — a complete e-commerce front-end that brings together everything: routing (Day 12), forms (Day 13), and a Redux cart (today), all in TypeScript.

> QUICK RECAP
>
> **useState** is local; **Redux** is one global store any component can use · reach for it when far-apart components share data · **Redux Toolkit** = `configureStore` + `createSlice` + `Provider` + typed hooks · flow: **dispatch → reducer → selector re-renders** · persist with a small **localStorage** adapter.